LAB ASSIGMENT-8

Function Template

1. Write a function template to swap two variables of any data type.

Code:

```
#include <iostream>
using namespace std;
template <class T>
void swapValues(T &a, T &b)
{
   T temp;
   temp = a;
   a = b;
   b = temp;
}
int main()
{
   int x = 10, y = 20;
   swapValues(x,y);
   cout << x << " " << y;
   return 0;
}
```

2. Write a function template to find the minimum element in an array.

Code:

```
#include <iostream>
using namespace std;
template <class T>
T minimum(T arr[], int size)
{
   T min = arr[0];
   for(int i = 1; i < size; i++)
   {
      if(arr[i] < min)
```

```
        {
            min = arr[i];
        }
    }
    return min;
}
int main()
{
    int arr[5] = {5,2,8,1,9};
    cout << "Minimum = "  << minimum(arr,5);
    return 0;
}
```

1. Write a function template to sort an array using bubble sort.

Code:

```
#include <iostream>
using namespace std;
template <class T>
void bubbleSort(T arr[], int size)
{
    for(int i = 0; i < size - 1; i++)
    {
        for(int j = 0; j < size - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                T temp;
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
```

```
}

int main()
{
   int arr[5] = {5,3,1,4,2};
   bubbleSort(arr,5);
   for(int i = 0; i < 5; i++)
   {
      cout << arr[i] << " ";
   }
   return 0;
}
```

2. Write a function template to perform linear search.

Code:

```
#include <iostream>
using namespace std;
template <class T>
int linearSearch(T arr[], int size, T key)
{
   for(int i = 0; i < size; i++)
   {
      if(arr[i] == key)
      {
         return i;
      }
   }
   return -1;
}

int main()
{
```

```
    int arr[5] = {10,20,30,40,50};
    int pos = linearSearch(arr,5,30);
    if(pos != -1)
    {
       cout << "Element Found at Index " << pos;
    }
    else
    {
       cout << "Element Not Found";
    }
    return 0;
}
```

3. Design overloaded versions of a function template process() such that:

   i)Accepts a single parameter

   ii)Accepts two parameters of the same type

   iii) Accepts two parameters of different types

Code:

```
#include <iostream>
using namespace std;
template <class T>
void process(T a)
{
   cout << "One Parameter: "
        << a << endl;
}
template <class T>
void process(T a, T b)
{
   cout << "Same Type Parameters: "  << a << "  " << b << endl;
}
template <class T1, class T2>
void process(T1 a, T2 b)
```

```
{
   cout << "Different Type Parameters: "  << a << "  " << b << endl;
}
int main()
{
   process(10);
   process(5,8);
   process(5,"Hello");
   return 0;
}
```

Class Template

1. Develop a class template for stack with push and pop operations.

Code:

```
#include <iostream>
using namespace std;
template <class T>
class Stack
{
private:
   T arr[5];
   int top;
public:
   Stack()
   {
      top = -1;
   }
   void push(T value)
   {
      if(top == 4)
      {
         cout << "Stack Overflow\n";
      }
      else
```

```
        {
            top++;
            arr[top] = value;
        }
    }
    void pop()
    {
        if(top == -1)
        {
            cout << "Stack Underflow";
        }
        else
        {
            cout << "Deleted: " << arr[top] << endl;
            top--;
        }
    }
};
int main()
{
    Stack<int> s;
    s.push(10);
    s.push(20);
    s.pop();
    return 0;
}
```

2. Develop a class template for queue with enqueue and dequeue operations.

Code:

```
#include <iostream>
using namespace std;
template <class T>
class Queue
```

```
{
private:
    T arr[5];
    int front, rear;
public:
    Queue()
    {
        front = 0;
        rear = -1;
    }
    void enqueue(T value)
    {
        if(rear == 4)
        {
            cout << "Queue Full";
        }
        else
        {
            rear++;
            arr[rear] = value;
        }
    }
    void dequeue()
    {
        if(front > rear)
        {
            cout << "Queue Empty";
        }
        else
        {
            cout << "Deleted: " << arr[front] << endl;
```

```
            front++;
        }
    }
};

int main()
{
    Queue<int> q;
    q.enqueue(10);
    q.enqueue(20);
    q.dequeue();
    return 0;
}
```

3. Create a class template to store and display a pair of values.

Code:

```
#include <iostream>
using namespace std;
template <class T>
class Pair
{
private:
    T a, b;
public:
    Pair(T x, T y)
    {
        a = x;
        b = y;
    }
    void display()
    {
        cout << "Values: " << a << " " << b;
```

```
    }
};
int main()
{
    Pair<int> p(10,20);
    p.display();
    return 0;
}
```

4. Create a class template for basic arithmetic operations.

Code:

```
#include <iostream>
using namespace std;
template <class T>
class Arithmetic
{
private:
    T a, b;
public:
    Arithmetic(T x, T y)
    {
        a = x;
        b = y;
    }
    void add()
    {
        cout << "Addition = "<< a + b << endl;
    }
    void subtract()
    {
        cout << "Subtraction = "<< a - b << endl;
    }
    void multiply()
```

```
    {
        cout << "Multiplication = "<< a * b << endl;
    }
};
int main()
{
    Arithmetic<int> a(10,5);
    a.add();
    a.subtract();
    a.multiply();
    return 0;
}
```

5. Create a class template to input and display array elements.

Code:

```
#include <iostream>
using namespace std;
template <class T>
class Array
{
private:
    T arr[5];
public:
    void input()
    {
        for(int i = 0; i < 5; i++)
        {
            cin >> arr[i];
        }
    }
    void display()
    {
        for(int i = 0; i < 5; i++)
```

```
        {
            cout << arr[i] << " ";
        }
    }
};
int main()
{
    Array<int> a;
    cout << "Enter Elements"<<endl;
    a.input();
    cout << "Array Elements"<<endl;
    a.display();
    return 0;
}
```